□> **Templates:** C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-en.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx

PDF: TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_01.pdf

Instructor: BELACEL Madani

Level: ENS â€" Year 2

Duration: 1h30 (90 min)

Learning Objectives

Practice OOP, algorithms, SQL, Web, Git and lesson plan design.

Exercise 1 â€" Python OOP *(18 min)*

Problem statement: Create a `Grade` class (subject, value, coefficient) and a `ReportCard` class computing the weighted average.

Solution:

```
class Grade:
def __init__(self, subject, value, coef=1):
self.subject = subject
self.value = value
self.coef = coef

class ReportCard:
def __init__(self, grades):
self.grades = grades
def average(self):
s = sum(g.value * g.coef for g in self.grades)
c = sum(g.coef for g in self.grades)
return s / c
```

Exercise 2 â€" Binary Search *(18 min)*

Problem statement: Binary search pseudocode. Precondition? Complexity?

Solution:

• Array must be **sorted**.

• Divide and conquer: compare middle, reduce interval.

• Complexity O(log n).

Exercise 3 â€" SQL *(18 min)*

Problem statement: Tables `student(id, name)` and `grade(student_id, subject, value)`. Query: average per student.

Solution:

```
SELECT s.name, AVG(g.value) AS average
FROM student s
JOIN grade g ON s.id = g.student_id
```

```
GROUP BY s.id, s.name;
```

Exercise 4 â€” Web Page *(18 min)*

Problem statement: HTML form (name, email) + centered CSS + JS alert on submission.

Solution:

• `<form>` with `input type="text"` and `email`.

• CSS `margin: auto; max-width: 400px`.

• `onsubmit` â†’ `preventDefault()` + `alert('Thank you')`.

Exercise 5 â€” Git and Lesson Plan *(13 min)*

Problem statement: Describe Git workflow for a pair project. Outline a 45-min session "Loops in Python" for 8th grade.

Solution:

• `main` + `feature` branches; atomic commits; merge request.

• Session: review 10 min, guided exercises 25 min, challenge 10 min; criteria: working code, readability.

Tutorial Plan â€” 90 min

Ex.1 18' | Ex.2 18' | Ex.3 18' | Ex.4 18' | Ex.5 13' | Closing 5'

Conceptual Diagrams

```
flowchart TD
A[TD Start] --> B[Exercise 1<br/>Python OOP]
B --> C[Exercise 2<br/>Binary Search]
C --> D[Exercise 3<br/>SQL]
D --> E[Exercise 4<br/>HTML/CSS/JS Web Page]
E --> F[Exercise 5<br/>Git & Pedagogy]
F --> G[Closing]

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#9b59b6,color:#fff
style F fill:#2ecc71,color:#fff
style G fill:#4a90d9,color:#fff
```

```
mindmap
root((CS Tutorial))
OOP
Grade Class
ReportCard Class
Weighted Average
Algorithms
Binary Search
O(log n) Complexity
Sorted Array
SQL
```

```
SELECT Query
JOIN
GROUP BY
Web
HTML Form
Centered CSS
JavaScript Alert
Git
Branches
Atomic Commits
Merge Request
```

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
OOP: class and average	20 %	Classes absent or incorrect	Partial class without computation	Correct classes with workin
Binary search	20 %	Algorithm not mastered	Partial logic	Correct implementation wit
SQL: join and aggregation	20 %	Incorrect query	Correct syntax but wrong result	Exact and optimal query
Web page: HTML/CSS/JS	20 %	No working output	Form without JS	Complete form with JS vali
Git and pedagogical design	20 %	Workflow unknown	Partial workflow	Full workflow and structure

Pedagogical Note ENS

Teaching transposition: This tutorial covers five fundamental pillars of computer science (OOP, algorithms, SQL, Web, Git) that a future middle/high school teacher must master in order to transpose these concepts into their technology courses. The variety of exercises shows students how the same principles (abstraction, structuring, automation) apply across different domains.

Trainer advice: Encourage students to explicitly link each exercise to secondary school curricula. For example, binary search can be reformulated as a "guess the number" game in middle school. Have them produce a reflective written record after each exercise.

Extension: Propose an interdisciplinary session where 8th graders create a mini-project combining a Web form, a database, and grade visualization with a chart.

Templates: `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-en.docx` | `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF: `TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_02.pdf`

Computer Science Tutorial â€” ENS Year 2 â€” Session 02 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 2 â€” English Department

Duration: 1h30

Academic year: 2025-2026

Exercise 1 â€” Defining a Class *(15 min)*

Problem statement: Create a `Book` class with:

• Attributes: `title`, `author`, `year`, `pages`

• `__init__` method with all parameters

• `display()` method that prints the book information

• `is_long()` method returning True if the book has more than 300 pages

Solution:

```
class Book:
def __init__(self, title, author, year, pages):
self.title = title
self.author = author
self.year = year
self.pages = pages

def display(self):
print(f"'{self.title}' â€” {self.author} ({self.year}), {self.pages}p.")

def is_long(self):
return self.pages > 300

# Test
b = Book("CS at ENS", "Belacel", 2026, 250)
b.display()
print("Long?", b.is_long()) # False
```

Exercise 2 â€” Attributes and Methods *(15 min)*

Problem statement: Complete the `BankAccount` class:

```
class BankAccount:
def __init__(self, holder, initial_balance=0):
# Complete: store the holder and the balance
pass

def deposit(self, amount):
# Complete: add the amount to the balance (check amount > 0)
pass

def withdraw(self, amount):
# Complete: withdraw if balance is sufficient
pass

def display(self):
# Complete: display the holder and the balance
```

```
pass
```

Solution:

```
class BankAccount:
def __init__(self, holder, initial_balance=0):
self.holder = holder
self.balance = initial_balance if initial_balance >= 0 else 0

def deposit(self, amount):
if amount > 0:
self.balance += amount
print(f"Deposit of {amount} â,¬ completed.")
else:
print("Invalid amount.")

def withdraw(self, amount):
if 0 < amount <= self.balance:
self.balance -= amount
print(f"Withdrawal of {amount} â,¬ completed.")
else:
print("Insufficient balance or invalid amount.")

def display(self):
print(f"Holder: {self.holder} â€" Balance: {self.balance} â,¬")
```

Exercise 3 â€" Class Variables *(15 min)*

Problem statement: Create a `Student` class with:

• Class variable `university = "ENS Mostaganem"`

• Class variable `total_students = 0`

• Instance attributes: `name`, `group`

• The constructor increments `total_students`

• `__str__` method returning a descriptive string

Create 3 students and display their info and the total.

Solution:

```
class Student:
university = "ENS Mostaganem"
total_students = 0

def __init__(self, name, group):
self.name = name
self.group = group
Student.total_students += 1

def __str__(self):
return f"{self.name} â€" {self.group} ({self.university})"

# Test
s1 = Student("Ali", "ENS2-A")
s2 = Student("Sara", "ENS2-A")
s3 = Student("Mohamed", "ENS2-B")

print(s1)
print(s2)
print(s3)
```

```
print(f"Total: {Student.total_students} students")
```

Exercise 4 â€” Class Composition *(20 min)*

Problem statement: Create two classes:

1. `Subject` with attributes `name`, `coefficient`

2. `Assessment` with attributes `subject` (Subject object), `grade`, `date`

• Method `weighted()` returning `grade Ã— coefficient`

Create a subject "Computer Science" (coef 3) and an assessment with grade 15. Display the weighted result.

Solution:

```
class Subject:
def __init__(self, name, coefficient):
self.name = name
self.coefficient = coefficient

def __str__(self):
return f"{self.name} (coef {self.coefficient})"

class Assessment:
def __init__(self, subject, grade, date):
self.subject = subject
self.grade = grade
self.date = date

def weighted(self):
return self.grade * self.subject.coefficient

def __str__(self):
return f"{self.subject.name}: {self.grade}/20 (coef {self.subject.coefficient}) â†’
{self.weighted()} pts"

# Test
cs = Subject("Computer Science", 3)
a1 = Assessment(cs, 15, "2026-03-15")
print(a1)
```

Exercise 5 â€” Class with Object List *(20 min)*

Problem statement: Create a `Pupil` class that contains a list of assessments. Add methods:

• `add_assessment(subject, grade, date)` â€” creates an Assessment and adds it to the list

• `average()` â€” computes the weighted average

• `show_report()` â€” displays all assessments and the average

Solution:

```
class Pupil:
def __init__(self, last_name, first_name):
self.last_name = last_name
self.first_name = first_name
self.assessments = []

def add_assessment(self, subject, grade, date):
```

```
a = Assessment(subject, grade, date)
self.assessments.append(a)

def average(self):
if not self.assessments:
return 0.0
total_weighted = sum(a.weighted() for a in self.assessments)
total_coef = sum(a.subject.coefficient for a in self.assessments)
return total_weighted / total_coef if total_coef > 0 else 0.0

def show_report(self):
print(f"=== Report for {self.first_name} {self.last_name} ===")
for a in self.assessments:
print(f" {a}")
print(f" Overall average: {self.average():.2f}/20")

# Test
french = Subject("French", 2)
english = Subject("English", 1)

pupil = Pupil("Benali", "Sara")
pupil.add_assessment(cs, 15, "2026-03-15")
pupil.add_assessment(french, 12, "2026-03-16")
pupil.add_assessment(english, 18, "2026-03-17")
pupil.show_report()
```

Session Plan â€” 1h30 (90 min)

Phase	Duration	Cumulative
Welcome	5 min	5 min
Exercise 1 â€” Book Class	15 min	20 min
Exercise 2 â€” BankAccount	15 min	35 min
Exercise 3 â€” Class Variables	15 min	50 min
Exercise 4 â€” Composition	20 min	70 min
Exercise 5 â€” Object List	20 min	90 min
Total	90 min	

Conceptual Diagrams

```
flowchart LR
A[Class] --> B[Instance<br/>Attributes]
A --> C[Class<br/>Attributes]
A --> D[Methods]
B --> E[Book<br/>BankAccount]
C --> F[university<br/>total_students]
D --> G[Composition]
G --> H[Subject + Assessment<br/>â†’ Pupil]

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#2ecc71,color:#fff
style D fill:#e67e22,color:#fff
style G fill:#e74c3c,color:#fff
style H fill:#9b59b6,color:#fff
```

```
classDiagram
class Book {
- title: string
- author: string
- year: int
- pages: int
+ display()
+ is_long() bool
}
class BankAccount {
- holder: string
- balance: float
+ deposit(amount)
+ withdraw(amount)
+ display()
}
class Student {
+ university: string
+ total_students: int
- name: string
- group: string
+ __str__() string
}
class Subject {
- name: string
- coefficient: int
}
class Assessment {
- subject: Subject
- grade: float
- date: string
+ weighted() float
}
class Pupil {
- last_name: string
- first_name: string
- assessments: list
+ add_assessment()
+ average() float
+ show_report()
}
Subject --> Assessment : composed in
Assessment --> Pupil : aggregated in

style Book fill:#4a90d9,color:#fff
style BankAccount fill:#2ecc71,color:#fff
style Student fill:#e67e22,color:#fff
style Subject fill:#e74c3c,color:#fff
style Assessment fill:#9b59b6,color:#fff
style Pupil fill:#4a90d9,color:#fff
```

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acqu
Class creation (Book)	20 %	Class absent	Attributes partially defined	Com
Attributes and methods (BankAccount)	20 %	Methods not implemented	Partial logic (deposit without check)	All m
Class variables (Student)	20 %	Variables not declared	Counter not functional	Class
Composition (Subject + Assessment)	20 %	Relationship not modeled	Partial composition without weighted method	Com

Object list (Pupil)	20 %	List not used	Non-weighted average	Full r

Pedagogical Note ENS

Teaching transposition: Class and object concepts can be transposed to middle school through the notion of a "template form": a class is a template of which each object is a concrete instance (e.g., student card template â†’ filled cards). Composition can be illustrated by a schoolbag containing several subjects.

Trainer advice: Have students create a paper poster representing a class with its attributes and methods, then ask them to "act out" the execution of a method in groups. This kinesthetic approach strengthens understanding of abstract concepts.

Extension: Propose a mini-project to create a library management system with Book, Member, and Loan classes, including data persistence in a JSON file.

Templates: `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-en.docx` | `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF: `TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_03.pdf`

Computer Science Tutorial â€” ENS Year 2 â€” Session 03 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 2 â€” English Department

Duration: 1h30

Academic year: 2025-2026

Exercise 1 â€” Simple Inheritance *(15 min)*

Problem statement: Create a parent class `Person` and a child class `Teacher`:

```
class Person:
def __init__(self, last_name, first_name, email):
# Complete
pass

def introduce(self):
return f"My name is {self.first_name} {self.last_name}"

class Teacher(Person):
def __init__(self, last_name, first_name, email, subject):
# Complete: call the parent constructor AND store the subject
pass

def introduce(self):
# Complete: override to add the subject
pass
```

Solution:

```
class Person:
def __init__(self, last_name, first_name, email):
self.last_name = last_name
self.first_name = first_name
self.email = email

def introduce(self):
return f"My name is {self.first_name} {self.last_name}"

class Teacher(Person):
def __init__(self, last_name, first_name, email, subject):
super().__init__(last_name, first_name, email)
self.subject = subject

def introduce(self):
base = super().introduce()
return f"{base} and I teach {self.subject}"

# Test
teacher = Teacher("Belacel", "Madani", "m.belacel@ens.dz", "Computer Science")
print(teacher.introduce())
```

Exercise 2 â€” Override and super() *(15 min)*

Problem statement: Create a three-level hierarchy:

1. `Animal`: attribute `name`, method `speak()` â†’ returns "..."

2. `Dog(Animal)`: overrides `speak()` â†' returns "Woof!"

3. `GuardDog(Dog)`: overrides `speak()` â†' uses `super().speak()` then adds " Bad dog!"

Create a GuardDog named "Rex" and call `speak()`.

Solution:

```
class Animal:
def __init__(self, name):
self.name = name

def speak(self):
return "..."

class Dog(Animal):
def speak(self):
return "Woof!"

class GuardDog(Dog):
def speak(self):
return super().speak() + " Bad dog!"

# Test
rex = GuardDog("Rex")
print(f"{rex.name} says: {rex.speak()}")
```

Exercise 3 â€" Polymorphism *(20 min)*

Problem statement: Create a function `make_speak(animal)` that calls the `speak()` method of any animal. Then create:

• `Cat(Animal)` with `speak()` â†' "Meow!"

• `Cow(Animal)` with `speak()` â†' "Moo!"

• `Duck(Animal)` with `speak()` â†' "Quack!"

Test the function with the three animals.

Solution:

```
class Cat(Animal):
def speak(self):
return "Meow!"

class Cow(Animal):
def speak(self):
return "Moo!"

class Duck(Animal):
def speak(self):
return "Quack!"

def make_speak(animal):
"""Polymorphic function: Accepts any Animal"""
print(f"{animal.name} says: {animal.speak()}")

# Test
animals = [
Cat("Kitty"),
Cow("Daisy"),
Duck("Donald"),
GuardDog("Rex")
]
```

```
for a in animals:
make_speak(a)
```

Output:

```
Kitty says: Meow!
Daisy says: Moo!
Donald says: Quack!
Rex says: Woof! Bad dog!
```

Exercise 4 â€” Encapsulation *(20 min)*

Problem statement: Create a `SavingsAccount` class with:

• Protected attribute `_rate` (annual interest rate in %, e.g., 2.5)

• Private attribute `__balance` (cannot be modified directly)

• Getter `balance` with `@property`

• Setter `balance` with `@balance.setter` (checks that amount â‰¥ 0)

• Method `apply_interest()` that adds interest to the balance

Solution:

```
class SavingsAccount:
def __init__(self, holder, balance=0, rate=2.5):
self.holder = holder
self._rate = rate
self.__balance = 0
if balance >= 0:
self.__balance = balance

@property
def balance(self):
return self.__balance

@balance.setter
def balance(self, amount):
if amount >= 0:
self.__balance = amount
else:
print("Error: balance cannot be negative.")

def apply_interest(self):
interest = self.__balance * (self._rate / 100)
self.__balance += interest
print(f"Interest of {interest:.2f} â,¬ applied (rate {self._rate}%).")

def __str__(self):
return f"{self.holder} â€” {self.__balance:.2f} â,¬"

# Test
sa = SavingsAccount("Ali", 1000, 2.5)
print(sa)
sa.apply_interest()
print(sa)
print(sa.balance)
```

Exercise 5 â€” Complete School Hierarchy *(20 min)*

Problem statement: Create the following classes with inheritance:

1. `ENS_Member`: last_name, first_name, email
2. `Student(ENS_Member)`: group, list of grades

• `add_grade(grade)`, `average()`, `__str__()`

3. `Professor(ENS_Member)`: subject, list of courses

• `add_course(course)`, `__str__()`

4. Test with one student and one professor

Solution:

```
class ENS_Member:
def __init__(self, last_name, first_name, email):
self.last_name = last_name
self.first_name = first_name
self.email = email

def __str__(self):
return f"{self.first_name} {self.last_name}"

class Student(ENS_Member):
def __init__(self, last_name, first_name, email, group):
super().__init__(last_name, first_name, email)
self.group = group
self.grades = []

def add_grade(self, grade):
if 0 <= grade <= 20:
self.grades.append(grade)

def average(self):
return sum(self.grades) / len(self.grades) if self.grades else 0.0

def __str__(self):
return f"{super().__str__()} â€” {self.group} (Avg: {self.average():.2f})"

class Professor(ENS_Member):
def __init__(self, last_name, first_name, email, subject):
super().__init__(last_name, first_name, email)
self.subject = subject
self.courses = []

def add_course(self, course):
self.courses.append(course)

def __str__(self):
return f"{super().__str__()} â€” {self.subject} ({len(self.courses)} courses)"

# Test
student = Student("Benali", "Sara", "s.benali@ens.dz", "ENS2-A")
student.add_grade(15)
student.add_grade(12)
student.add_grade(18)
print(student)

prof = Professor("Belacel", "Madani", "m.belacel@ens.dz", "Computer Science")
prof.add_course("OOP")
prof.add_course("DB")
print(prof)
```

Session Plan â€” 1h30 (90 min)

Phase	Duration	Cumulative
Welcome	5 min	5 min
Exercise 1 â€” Simple Inheritance	15 min	20 min
Exercise 2 â€” Override and super()	15 min	35 min
Exercise 3 â€” Polymorphism	20 min	55 min
Exercise 4 â€” Encapsulation	20 min	75 min
Exercise 5 â€” Complete Hierarchy	15 min	90 min
Total	90 min	

Conceptual Diagrams

```
flowchart TD
A[Inheritance] --> B[Parent class<br/>Person / Animal]
A --> C[Override<br/>super()]
A --> D[Polymorphism]
A --> E[Encapsulation]
D --> F[make_speak()<br/>one interface]
E --> G[Private attributes<br/>__balance]
E --> H[Getters/Setters<br/>@property]
B --> I[Complete hierarchy<br/>ENS_Member â†’ Student / Professor]

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#2ecc71,color:#fff
style D fill:#e67e22,color:#fff
style E fill:#e74c3c,color:#fff
style I fill:#9b59b6,color:#fff

classDiagram
class Animal {
- name: string
+ speak() string
}
class Dog {
+ speak() string
}
class GuardDog {
+ speak() string
}
class Cat {
+ speak() string
}
class Cow {
+ speak() string
}
class Duck {
+ speak() string
}
Animal <|-- Dog
Animal <|-- Cat
Animal <|-- Cow
Animal <|-- Duck
Dog <|-- GuardDog

class ENS_Member {
```

```
- last_name: string
- first_name: string
- email: string
}
class Student {
- group: string
- grades: list
+ add_grade(grade)
+ average() float
}
class Professor {
- subject: string
- courses: list
+ add_course(course)
}
ENS_Member <|-- Student
ENS_Member <|-- Professor

style Animal fill:#4a90d9,color:#fff
style Dog fill:#2ecc71,color:#fff
style GuardDog fill:#e67e22,color:#fff
style Cat fill:#e74c3c,color:#fff
style Cow fill:#9b59b6,color:#fff
style Duck fill:#4a90d9,color:#fff
style ENS_Member fill:#2ecc71,color:#fff
style Student fill:#e67e22,color:#fff
style Professor fill:#e74c3c,color:#fff
```

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquir
Simple inheritance (Person â†’ Teacher)	20 %	Inheritance not understood	super() call missing	Correct
Override and super() (Animal â†’ GuardDog)	20 %	Inheritance chain absent	super() misused	Full hie
Polymorphism (make_speak)	20 %	Concept not mastered	Function without polymorphic typing	Polymo
Encapsulation (SavingsAccount)	20 %	All attributes public	Property without setter	Full en
Complete hierarchy (ENS_Member)	20 %	No hierarchy	Partial hierarchy	Full hie

Pedagogical Note ENS

Teaching transposition: Inheritance can be introduced in middle school through the notion of "categories and subcategories" (e.g., Animal â†’ Mammal â†’ Dog), which corresponds to the classification thinking taught in science. Polymorphism is illustrated by a single instruction ("introduce yourself") producing different results depending on the object.

Trainer advice: Use visual aids (class cards, inheritance diagrams on the board) before moving to code. Offer an unplugged exercise where students embody objects and execute methods by following the inheritance chain.

Extension: Study the "Template Method" or "Strategy" design pattern, showing how inheritance and polymorphism enable the design of reusable frameworks.

Templates: `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-en.docx` | `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF: `TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_04.pdf`

Computer Science Tutorial â€” ENS Year 2 â€” Session 04 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 2 â€” English Department

Duration: 1h30

Academic year: 2025-2026

Exercise 1 â€” Recursion: Factorial *(15 min)*

Problem statement: Write a recursive function `factorial(n)` and trace its execution for n=4.

Solution:

```
def factorial(n):
if n <= 1: # Base case
return 1
return n * factorial(n - 1) # Recursive call
```

Trace for n = 4:

```
factorial(4)
â†’ 4 * factorial(3)
â†’ 4 * (3 * factorial(2))
â†’ 4 * (3 * (2 * factorial(1)))
â†’ 4 * (3 * (2 * 1))
â†’ 4 * (3 * 2)
â†’ 4 * 6
â†’ 24
```

Exercise 2 â€” Recursion: Fibonacci *(20 min)*

Problem statement: Write a recursive function `fibonacci(n)` that returns the n-th Fibonacci term (Fâ‚€=0, Fâ‚=1, Fâ‚™=Fâ‚™â‚‹â‚+Fâ‚™â‚‹â‚‚).

Compute `fibonacci(7)` by hand.

Solution:

```
def fibonacci(n):
if n <= 1: # Base case: Fâ‚€=0, Fâ‚=1
return n
return fibonacci(n-1) + fibonacci(n-2) # Double call
```

Values:

```
fibonacci(0) = 0
fibonacci(1) = 1
fibonacci(2) = 1
fibonacci(3) = 2
fibonacci(4) = 3
fibonacci(5) = 5
fibonacci(6) = 8
fibonacci(7) = 13
```

Limitation: this recursive version is very slow (O(2â¿)). For n=40, it makes billions of calls.

Exercise 3 â€” Binary Search *(20 min)*

Problem statement: Write a function `binary_search(array, target)` that searches for an element in a sorted array.

Apply to: `grades = [5, 8, 10, 12, 15, 18, 20]` to find 15.

Solution:

```
def binary_search(array, target):
low = 0
high = len(array) - 1

while low <= high:
mid = (low + high) // 2
if array[mid] == target:
return mid
elif array[mid] < target:
low = mid + 1 # Search right
else:
high = mid - 1 # Search left

return -1 # Not found

# Test
grades = [5, 8, 10, 12, 15, 18, 20]
pos = binary_search(grades, 15)
print(f"15 found at index {pos}") # 4

# Comparison: linear search O(n) vs binary search O(log n)
# n = 1,000,000: linear = 1M operations, binary = ~20 operations
```

Exercise 4 â€” Bubble Sort *(20 min)*

Problem statement: Write a function `bubble_sort(array)` that sorts an array in ascending order.

Trace the execution on: `[15, 8, 12, 20, 5]`

Solution:

```
def bubble_sort(array):
n = len(array)
for i in range(n):
swapped = True
for j in range(n - i - 1):
if array[j] > array[j + 1]:
array[j], array[j + 1] = array[j + 1], array[j]
swapped = False
if swapped:
break
return array
```

Trace:

```
Start: [15, 8, 12, 20, 5]
Pass 1: [8, 12, 15, 5, 20] (15 sinks to bottom)
Pass 2: [8, 12, 5, 15, 20] (12 rises)
Pass 3: [8, 5, 12, 15, 20] (8 rises)
Pass 4: [5, 8, 12, 15, 20] (sorted)
```

Complexity: O(nÂ²) in worst case (reversed array), O(n) if already sorted.

Exercise 5 â€” Merge Sort *(15 min)*

Problem statement: Write the functions `merge_sort(array)` and `merge(left, right)`.

Sort `[15, 8, 12, 20, 5, 10, 18]` using merge sort.

Solution:

```
def merge_sort(array):
if len(array) <= 1:
return array

mid = len(array) // 2
left = merge_sort(array[:mid])
right = merge_sort(array[mid:])

return merge(left, right)

def merge(left, right):
result = []
i = j = 0
while i < len(left) and j < len(right):
if left[i] <= right[j]:
result.append(left[i])
i += 1
else:
result.append(right[j])
j += 1
result.extend(left[i:])
result.extend(right[j:])
return result

# Test
grades = [15, 8, 12, 20, 5, 10, 18]
print(f"Before: {grades}")
sorted_grades = merge_sort(grades)
print(f"After: {sorted_grades}")
```

Split trace:

```
[15, 8, 12, 20, 5, 10, 18]
â”œâ”€€ [15, 8, 12, 20]
â”, â”œâ”€€ [15, 8]
â”, â”, â”œâ”€€ [15]
â”, â”, â”"â”€€ [8] â†' merge â†' [8, 15]
â”, â”"â”€€ [12, 20]
â”, â”œâ”€€ [12]
â”, â”"â”€€ [20] â†' merge â†' [12, 20]
â”, â†' merge â†' [8, 12, 15, 20]
â”"â”€€ [5, 10, 18]
â”œâ”€€ [5]
â”"â”€€ [10, 18]
â”œâ”€€ [10]
â”"â”€€ [18] â†' merge â†' [10, 18]
â†' merge â†' [5, 10, 18]
â†' merge â†' [5, 8, 10, 12, 15, 18, 20]
```

Complexity: O(n log n) in all cases. More efficient than bubble sort for large arrays.

Session Plan â€" 1h30 (90 min)

Phase	Duration	Cumulative
Welcome	5 min	5 min
Exercise 1 â€" Factorial	15 min	20 min

Exercise 2 â€” Fibonacci	20 min	40 min
Exercise 3 â€” Binary Search	20 min	60 min
Exercise 4 â€” Bubble Sort	20 min	80 min
Exercise 5 â€” Merge Sort	10 min	90 min
Total	90 min	

Conceptual Diagrams

```
flowchart TD
A[Search and Sort Algorithms] --> B[Recursion]
A --> C[Search]
A --> D[Sorting]
B --> E[Factorial<br/>recursive]
B --> F[Fibonacci<br/>recursive]
C --> G[Binary search<br/>O(log n)]
D --> H[Bubble sort<br/>O(nÂ²)]
D --> I[Merge sort<br/>O(n log n)]

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style I fill:#9b59b6,color:#fff

flowchart LR
subgraph Recursion
F1[factorial(4)] --> F2[4 Ã— factorial(3)]
F2 --> F3[3 Ã— factorial(2)]
F3 --> F4[2 Ã— factorial(1)]
F4 --> F5[1]
end

subgraph Comparison
B1[Bubble sort<br/>O(nÂ²)] --> B2["n=100: 10,000 ops"]
M1[Merge sort<br/>O(n log n)] --> M2["n=100: ~460 ops"]
end

style F1 fill:#4a90d9,color:#fff
style F5 fill:#2ecc71,color:#fff
style B1 fill:#e74c3c,color:#fff
style M1 fill:#2ecc71,color:#fff
```

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Recursion: factorial	20 %	Function not recursive	Base case poorly defined	Correct recursion with trace
Recursion: Fibonacci	20 %	Double call not understood	Partial results	Correct implementation and complex
Binary search	20 %	Algorithm not working	Partial logic (infinite loop)	Correct search with indices
Bubble sort	20 %	Sort not working	Algorithm without optimization	Sort with optimization (flag) and trac
Merge sort	20 %	Merge not implemented	Recursive split only	Complete merge sort with complexity

Pedagogical Note ENS

Teaching transposition: Recursion can be introduced in middle school through the image of "Russian dolls" or the repetition of a pattern within a pattern (fractals). Sorting algorithms (bubble, merge) can be simulated with numbered cards that students move on the table, enabling kinesthetic understanding of the mechanisms.

Trainer advice: After each algorithm, ask students to run it "by hand" on a small dataset before coding. Emphasize time complexity comparisons to develop a culture of algorithmic efficiency from initial training.

Extension: Offer an optimization challenge: implement an iterative version of Fibonacci (dynamic programming) and compare execution times with the naive recursive version for n=30, 35, 40.

Templates: `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-en.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF: `TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_05.pdf`

Computer Science Tutorial â€” ENS Year 2 â€” Session 05 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 2 â€” English Department

Duration: 1h30

Academic year: 2025-2026

Exercise 1 â€” CREATE TABLE and INSERT *(20 min)*

Problem statement: Consider the relational schema for a library:

• `Author (author_id INT PRIMARY KEY, last_name VARCHAR(50), first_name VARCHAR(50), birth_year INT)`

• `Book (book_id INT PRIMARY KEY, title VARCHAR(100), publication_year INT, author_id INT REFERENCES Author(author_id))`

1. Write the CREATE TABLE statements

2. Insert 3 authors and 4 books

3. Display all books

Solution:

```
CREATE TABLE Author (
author_id INT PRIMARY KEY,
last_name VARCHAR(50),
first_name VARCHAR(50),
birth_year INT
);

CREATE TABLE Book (
book_id INT PRIMARY KEY,
title VARCHAR(100),
publication_year INT,
author_id INT REFERENCES Author(author_id)
);

INSERT INTO Author VALUES
(1, 'Dijkstra', 'Edsger', 1930),
(2, 'Knuth', 'Donald', 1938),
(3, 'Turing', 'Alan', 1912);

INSERT INTO Book VALUES
(1, 'The Art of Computer Programming', 1968, 2),
(2, 'A Discipline of Programming', 1976, 1),
(3, 'Computing Machinery and Intelligence', 1950, 3),
(4, 'Selected Writings on Computing', 1982, 1);

SELECT * FROM Book;
```

Exercise 2 â€” SELECT and WHERE *(15 min)*

Problem statement: On the `Book` table above, write the following queries:

1. Titles of books published after 1970

2. Books by author_id = 1 (Dijkstra)

3. Books whose title contains "Programming"

Solution:

```
-- 1. Books after 1970
SELECT title, publication_year
FROM Book
WHERE publication_year > 1970;

-- 2. Books by Dijkstra
SELECT title
FROM Book
WHERE author_id = 1;

-- 3. Title contains "Programming"
SELECT title, publication_year
FROM Book
WHERE title LIKE '%Programming%';
```

Exercise 3 â€" JOIN with INNER JOIN *(20 min)*

Problem statement: Enrich the database with a `Loan` table `(loan_id INT PRIMARY KEY, book_id INT REFERENCES Book, loan_date DATE, borrower_name VARCHAR(50))`.

Insert 5 loans then write the queries:

1. Display each loan with the book title and the author's name

2. Count the number of loans per author

3. Find the book(s) never loaned

Solution:

```
CREATE TABLE Loan (
loan_id INT PRIMARY KEY,
book_id INT REFERENCES Book(book_id),
loan_date DATE,
borrower_name VARCHAR(50)
);

INSERT INTO Loan VALUES
(1, 1, '2026-01-15', 'Ali'),
(2, 1, '2026-02-10', 'Sara'),
(3, 3, '2026-03-05', 'Mohamed'),
(4, 2, '2026-03-12', 'Ali'),
(5, 4, '2026-04-01', 'Sara');

-- 1. Loans with title and author
SELECT l.borrower_name, b.title, a.last_name || ' ' || a.first_name AS author, l.loan_date
FROM Loan l
INNER JOIN Book b ON l.book_id = b.book_id
INNER JOIN Author a ON b.author_id = a.author_id;

-- 2. Number of loans per author
SELECT a.last_name || ' ' || a.first_name AS author, COUNT(l.loan_id) AS loan_count
FROM Author a
LEFT JOIN Book b ON a.author_id = b.author_id
LEFT JOIN Loan l ON b.book_id = l.book_id
GROUP BY a.author_id, author;

-- 3. Books never loaned
SELECT b.title
```

```
FROM Book b
LEFT JOIN Loan l ON b.book_id = l.book_id
WHERE l.loan_id IS NULL;
```

Exercise 4 â€” Update and Delete *(15 min)*

Problem statement: Write SQL queries to:

1. Add a `available BOOLEAN` column to Book
2. Update the column: TRUE if the book has no current loan
3. Delete an author and their books (CASCADE)

Solution:

```
-- 1. Add column
ALTER TABLE Book ADD COLUMN available BOOLEAN DEFAULT TRUE;

-- 2. Update (books with loans = not available)
UPDATE Book SET available = FALSE
WHERE book_id IN (SELECT DISTINCT book_id FROM Loan);

-- Verification
SELECT title, available FROM Book;

-- 3. Delete with CASCADE (if constraint declared ON DELETE CASCADE)
-- ALTER TABLE Book ADD CONSTRAINT fk_author
-- FOREIGN KEY (author_id) REFERENCES Author(author_id) ON DELETE CASCADE;
-- DELETE FROM Author WHERE author_id = 3;
```

Exercise 5 â€” Complete Modeling *(20 min)*

Problem statement: Design the relational schema for managing students and their course enrollments with grades:

1. Create tables `Student`, `Course`, `Enrollment`
2. Insert 3 students, 2 courses, 5 enrollments
3. For each student display: name, courses taken, and grade

Solution:

```
CREATE TABLE Student (
student_id INT PRIMARY KEY,
last_name VARCHAR(50),
first_name VARCHAR(50),
email VARCHAR(100)
);

CREATE TABLE Course (
course_id INT PRIMARY KEY,
title VARCHAR(100),
credits INT
);

CREATE TABLE Enrollment (
student_id INT REFERENCES Student(student_id),
course_id INT REFERENCES Course(course_id),
grade DECIMAL(4,2),
PRIMARY KEY (student_id, course_id)
```

```
);

INSERT INTO Student VALUES
(1, 'Benali', 'Sara', 's.benali@ens.dz'),
(2, 'Ziani', 'Ahmed', 'a.ziani@ens.dz'),
(3, 'Kaci', 'Leila', 'l.kaci@ens.dz');

INSERT INTO Course VALUES
(101, 'OOP', 4),
(102, 'DB', 3);

INSERT INTO Enrollment VALUES
(1, 101, 15.00),
(1, 102, 12.50),
(2, 101, 17.00),
(2, 102, 14.00),
(3, 101, 10.50);

-- Full report
SELECT s.last_name || ' ' || s.first_name AS student,
c.title AS course,
e.grade
FROM Enrollment e
JOIN Student s ON e.student_id = s.student_id
JOIN Course c ON e.course_id = c.course_id
ORDER BY student, course;

-- Average per student
SELECT s.last_name || ' ' || s.first_name AS student,
ROUND(AVG(e.grade), 2) AS average
FROM Enrollment e
JOIN Student s ON e.student_id = s.student_id
GROUP BY s.student_id, student;
```

Session Plan â€” 1h30 (90 min)

Phase	Duration	Cumulative
Welcome	5 min	5 min
Exercise 1 â€” CREATE / INSERT	20 min	25 min
Exercise 2 â€” SELECT / WHERE	15 min	40 min
Exercise 3 â€” Joins	20 min	60 min
Exercise 4 â€” UPDATE / DELETE	15 min	75 min
Exercise 5 â€” Modeling	15 min	90 min
Total	90 min	

Conceptual Diagrams

```
flowchart TD
A[Relational Database] --> B[Creation<br/>CREATE TABLE]
A --> C[Insertion<br/>INSERT]
A --> D[Querying<br/>SELECT]
A --> E[Update<br/>UPDATE / ALTER]
A --> F[Deletion<br/>DELETE / CASCADE]
D --> G[Joins<br/>INNER / LEFT JOIN]
```

```
D --> H[Filtering<br/>WHERE / LIKE]
D --> I[Aggregation<br/>GROUP BY / COUNT]

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#2ecc71,color:#fff
style D fill:#e67e22,color:#fff
style E fill:#e74c3c,color:#fff
style F fill:#e74c3c,color:#fff
style G fill:#9b59b6,color:#fff
```

```
erDiagram
Author ||--o{ Book : writes
Book ||--o{ Loan : concerns
Student ||--o{ Enrollment : performs
Course ||--o{ Enrollment : contains

Author {
int author_id PK
string last_name
string first_name
int birth_year
}
Book {
int book_id PK
string title
int publication_year
int author_id FK
}
Loan {
int loan_id PK
int book_id FK
date loan_date
string borrower_name
}
Student {
int student_id PK
string last_name
string first_name
string email
}
Course {
int course_id PK
string title
int credits
}
Enrollment {
int student_id FK
int course_id FK
decimal grade
}

style Author fill:#4a90d9,color:#fff
style Book fill:#2ecc71,color:#fff
style Loan fill:#e67e22,color:#fff
style Student fill:#9b59b6,color:#fff
style Course fill:#e74c3c,color:#fff
style Enrollment fill:#4a90d9,color:#fff
```

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Table creation and insertion	20 %	Tables not created	INSERT incomplete	Correct creation and insertion
SELECT and WHERE filtering	20 %	SELECT syntax unknown	Partial filtering	Correct queries with WHERE
Joins (INNER / LEFT JOIN)	20 %	JOIN not understood	INNER JOIN only	Correct multiple joins with ag
Update and deletion	20 %	ALTER / UPDATE unknown	UPDATE without subquery	UPDATE, ALTER and DELE
Complete modeling	20 %	Schema not normalized	Tables without foreign keys	Full schema with keys, insert

Pedagogical Note ENS

Teaching transposition: Relational modeling can be introduced in middle school by creating two-way tables (rows = records, columns = attributes), which students already use in mathematics. Joins can be illustrated by a "crossword puzzle" game where two grids are linked by a common piece of information.

Trainer advice: Start with a "paper-and-pencil" modeling exercise before moving to SQL. Ask students to draw the entity-relationship diagram on the board and verify cardinalities. Emphasize the difference between WHERE (row filtering) and HAVING (group filtering).

Extension: Propose a project to create a small database for the school library with a simple Web interface in PHP or Python Flask allowing users to list, add, and search for books.

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_06.pdf`

Computer Science Tutorial â€” ENS Year 2 â€” Session 06 (EN)

Lecturer: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 2 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Exercise 1 â€” Basic HTML Structure *(15 min)*

Problem statement: Create an HTML page `presentation.html` that presents your online CV. Include:

• Title "My CV"

• Header with your name (`<h1>` tag)

• A fictional profile picture (placeholder image)

• An "About" section in a paragraph

• A skills list (unordered)

• A footer with your email

Solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My CV</title>
</head>
<body>
<header>
<h1>Sara Benali</h1>
<img src="https://via.placeholder.com/150" alt="Profile picture">
</header>

<section>
<h2>About</h2>
<p>Second-year student at ENS Mostaganem. Passionate about computer science and
pedagogy.</p>
</section>

<section>
<h2>Skills</h2>
<ul>
<li>Python, Java, SQL</li>
<li>Web Development (HTML, CSS, JavaScript)</li>
<li>Operating Systems (Linux, Windows)</li>
</ul>
</section>

<footer>
<p>Contact: s.benali@ens.dz</p>
</footer>
</body>
</html>
```

Exercise 2 â€” CSS Layout *(20 min)*

Problem statement: Add an internal `style.css` to the previous page to:

• Center content with max-width 800px

• Change the header background color (light blue)

• Style sections with border and margins

• Use a Google Font (Roboto)

Solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My CV</title>
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap"
rel="stylesheet">
<style>
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}

body {
font-family: 'Roboto', sans-serif;
max-width: 800px;
margin: 0 auto;
padding: 20px;
background-color: #f5f5f5;
}

header {
background-color: #4a90d9;
color: white;
padding: 30px;
text-align: center;
border-radius: 10px;
margin-bottom: 20px;
}

header img {
border-radius: 50%;
margin-top: 10px;
}

section {
background-color: white;
border: 1px solid #ddd;
border-radius: 8px;
padding: 20px;
margin-bottom: 20px;
box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

h2 {
color: #4a90d9;
margin-bottom: 10px;
}

ul {
list-style-type: square;
```

```
padding-left: 20px;
}

footer {
text-align: center;
color: #888;
font-size: 0.9em;
}
</style>
</head>
<body>
<header>
<h1>Sara Benali</h1>
<img src="https://via.placeholder.com/150" alt="Profile picture">
</header>

<section>
<h2>About</h2>
<p>Second-year student at ENS Mostaganem.</p>
</section>

<section>
<h2>Skills</h2>
<ul>
<li>Python, Java, SQL</li>
<li>Web Development (HTML, CSS, JavaScript)</li>
<li>Operating Systems</li>
</ul>
</section>

<footer>
<p>Contact: s.benali@ens.dz</p>
</footer>
</body>
</html>
```

Exercise 3 â€” JavaScript: DOM and Events *(20 min)*

Problem statement: Add a JavaScript script to the CV page to:

1. A "Show/Hide" button for skills (toggle)
2. A visit counter stored in `localStorage`
3. Display the current date and time in the footer

Solution:

```
<script>
// 1. Toggle skills
function toggleSkills() {
const section = document.querySelector('section:nth-of-type(2)');
if (section.style.display === 'none') {
section.style.display = 'block';
btn.textContent = 'Hide skills';
} else {
section.style.display = 'none';
btn.textContent = 'Show skills';
}
}

const btn = document.createElement('button');
btn.textContent = 'Hide skills';
```

```
btn.onclick = toggleSkills;
document.body.insertBefore(btn, document.querySelector('footer'));

// 2. Visit counter
let visits = localStorage.getItem('cv_visits') || 0;
visits++;
localStorage.setItem('cv_visits', visits);
const counter = document.createElement('p');
counter.textContent = `Visits: ${visits}`;
document.querySelector('footer').appendChild(counter);

// 3. Clock
function displayTime() {
const now = new Date();
const clock = document.getElementById('clock');
if (clock) {
clock.textContent = now.toLocaleString('en-US');
}
}
const clock = document.createElement('p');
clock.id = 'clock';
document.querySelector('footer').appendChild(clock);
setInterval(displayTime, 1000);
displayTime();
</script>
```

Exercise 4 â€” Form with Validation *(20 min)*

Problem statement: Create an HTML page `contact.html` with a contact form:

• Fields: name, email, message

• JavaScript validation: name not empty, valid email (contains @), message â‰¥ 10 characters

• Display errors under each field

• On success, display "Thank you!" in a box

Solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Contact</title>
<style>
body { font-family: Arial, sans-serif; max-width: 600px; margin: 50px auto; }
label { display: block; margin-top: 10px; font-weight: bold; }
input, textarea { width: 100%; padding: 8px; margin-top: 5px; }
.error { color: red; font-size: 0.9em; }
.success { background: #d4edda; padding: 20px; border-radius: 5px; }
button { margin-top: 15px; padding: 10px 20px; background: #4a90d9; color: white; border:
none; cursor: pointer; }
</style>
</head>
<body>
<h1>Contact Us</h1>
<form id="contactForm">
<label>Name:</label>
<input type="text" id="name">
<div class="error" id="nameError"></div>

<label>Email:</label>
```

```
<input type="email" id="email">
<div class="error" id="emailError"></div>

<label>Message:</label>
<textarea id="message" rows="5"></textarea>
<div class="error" id="messageError"></div>

<button type="submit">Send</button>
</form>
<div id="success" style="display:none;" class="success"><h2>Thank you!</h2></div>

<script>
document.getElementById('contactForm').onsubmit = function(e) {
e.preventDefault();

// Reset
document.querySelectorAll('.error').forEach(el => el.textContent = '');
document.getElementById('success').style.display = 'none';

const name = document.getElementById('name').value.trim();
const email = document.getElementById('email').value.trim();
const message = document.getElementById('message').value.trim();
let valid = true;

if (!name) {
document.getElementById('nameError').textContent = 'Name is required.';
valid = false;
}

if (!email || !email.includes('@')) {
document.getElementById('emailError').textContent = 'Invalid email.';
valid = false;
}

if (message.length < 10) {
document.getElementById('messageError').textContent = 'Message must be at least 10
characters.';
valid = false;
}

if (valid) {
document.getElementById('contactForm').reset();
document.getElementById('success').style.display = 'block';
}
};
</script>
</body>
</html>
```

Exercise 5 â€” Mini-project: Interactive To-do List *(15 min)*

Problem statement: Create a to-do list with HTML + CSS + JS in a single file. Features:

• Input field + "Add" button

• Click on an item marks it as done (strikethrough)

• Delete button next to each task

Solution:

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
<meta charset="UTF-8">
<title>To-Do List</title>
<style>
body { font-family: Arial; max-width: 500px; margin: 50px auto; }
.add { display: flex; gap: 10px; margin-bottom: 20px; }
input { flex: 1; padding: 10px; }
button { padding: 10px 20px; cursor: pointer; }
ul { list-style: none; padding: 0; }
li { display: flex; justify-content: space-between; align-items: center;
padding: 10px; background: #f9f9f9; margin: 5px 0; border-radius: 5px; cursor: pointer; }
li.done { text-decoration: line-through; color: #888; background: #e9e9e9; }
.delete { background: #dc3545; color: white; border: none; padding: 5px 10px; cursor:
pointer; border-radius: 3px; }
</style>
</head>
<body>
<h1>To-Do List</h1>
<div class="add">
<input type="text" id="taskInput" placeholder="New task...">
<button onclick="addTask()">Add</button>
</div>
<ul id="taskList"></ul>

<script>
function addTask() {
const input = document.getElementById('taskInput');
const text = input.value.trim();
if (!text) return;

const li = document.createElement('li');
li.textContent = text;

li.onclick = function(e) {
if (e.target !== btn) this.classList.toggle('done');
};

const btn = document.createElement('button');
btn.textContent = 'âœ•';
btn.className = 'delete';
btn.onclick = function(e) {
e.stopPropagation();
li.remove();
};

li.appendChild(btn);
document.getElementById('taskList').appendChild(li);
input.value = '';
input.focus();
}

document.getElementById('taskInput').addEventListener('keypress', function(e) {
if (e.key === 'Enter') addTask();
});
</script>
</body>
</html>
```

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)

HTML Structure (titles, sections, footer)	20 %	Missing required elements	Partially complete	Complete and valid structure
CSS Layout	20 %	No styles applied	Partial styling	Coherent and responsive desig
JavaScript Interactivity (DOM)	20 %	No script	Partial scripts	Toggle, counter, clock working
Form Validation	20 %	No validation	Partial validation	Full validation with error messa
Interactive To-do List	20 %	Not functional	Partially functional	Add, mark, delete working

Pedagogical Note ENS

Teaching transposition: HTML/CSS/JS concepts can be transposed to middle school by first introducing web page structure with simple HTML, then styling with CSS without JavaScript. In high school, JavaScript interactivity can be added to create dynamic pages and forms.

Trainer advice: Emphasize separation of concerns (structure/style/behavior) and accessibility best practices. Offer progressive exercises: first reproduce mockups, then create freely.

Extension: Ask students to integrate an external API (e.g., weather) or publish their CV online via GitHub Pages.

Conceptual Diagrams

Diagram 1 â€" CV Page Structure

```
graph TD
A[CV Page] --> B[Header]
A --> C[About Section]
A --> D[Skills Section]
A --> E[Footer]
B --> F[h1 Title]
B --> G[Profile Picture]
D --> H[Python]
D --> I[Java]
D --> J[SQL]
D --> K[Web]
style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#9b59b6,color:#fff
```

Diagram 2 â€" Web Technologies

```
mindmap
root((Web Development))
HTML
Structure
Tags
Semantics
CSS
Layout
Colors
Fonts
Responsive
JavaScript
DOM
Events
localStorage
Forms
Tools
Browser
Code Editor
```

```
GitHub Pages
```

Session Schedule â€” 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome	5 min	5 min
Exercise 1 â€” HTML Structure	15 min	20 min
Exercise 2 â€” CSS	20 min	40 min
Exercise 3 â€” JavaScript DOM	20 min	60 min
Exercise 4 â€” Form Validation	20 min	80 min
Exercise 5 â€” To-do List	10 min	90 min
Total	90 min	

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_07.pdf`

Computer Science Tutorial â€” ENS Year 2 â€” Session 07 (EN)

Lecturer: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 2 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Exercise 1 â€” Initialization and First Commit *(15 min)*

Problem statement: Simulate Git commands to:

1. Initialize a Git repository
2. Create a `README.md` file
3. Add the file to the staging area
4. Make a commit with the message "Initial commit"

Solution:

```
# 1. Initialize the repository
git init

# 2. Create the file
echo "# ENS2 Project" > README.md
echo "Git Tutorial - Pedagogical Cycle" >> README.md

# 3. Add to staging
git add README.md

# 4. First commit
git commit -m "Initial commit"

# Verify
git log --oneline
```

Tree after Exercise 1:

```
* (HEAD -> master) Initial commit
```

Exercise 2 â€” Working with Branches *(20 min)*

Problem statement: Simulate the following workflow:

1. Create a branch `development`
2. Switch to that branch
3. Create `TP1.md` and `TP2.md` (each with one line of content)
4. Add and commit both files (message: "Add TP1 and TP2")
5. Switch back to `master` â€” notice the files are not there

Solution:

```
# 1-2. Create and switch
git checkout -b development
```

```
# 3. Create files
echo "# TP1 - OOP" > TP1.md
echo "# TP2 - DB" > TP2.md

# 4. Commit
git add TP1.md TP2.md
git commit -m "Add TP1 and TP2"

# 5. Back to master
git checkout master

# Verify
ls # TP1.md and TP2.md are NOT visible
git log --oneline --all # Shows both commits
```

Tree after Exercise 2:

```
* (development) Add TP1 and TP2
* (HEAD -> master) Initial commit
```

Exercise 3 â€” Merge and Conflict *(20 min)*

Problem statement: Simulate a merge conflict:

1. On `master`, modify `README.md`: add the line "## Description"
2. Commit: `git add . && git commit -m "Modify README on master"`
3. On `development`, modify `README.md`: add the line "## Author: Sara"
4. Commit: `git add . && git commit -m "Modify README on dev"`
5. Merge `development` into `master` â†' CONFLICT
6. Resolve the conflict manually

Solution:

```
# On master
echo -e "# ENS2 Project\n## Description" > README.md
git add README.md && git commit -m "Modify README on master"

# On development
git checkout development
echo -e "# ENS2 Project\n## Author: Sara" > README.md
git add README.md && git commit -m "Modify README on dev"

# Merge
git checkout master
git merge development
# CONFLICT in README.md

# Resolve: edit README.md to combine both
# Final content:
: # ENS2 Project
: ## Description
: ## Author: Sara

git add README.md
git commit -m "Resolve README conflict"
```

Tree after Exercise 3:

```
* (HEAD -> master) Resolve README conflict
|\
| * Modify README on dev
* | Modify README on master
|/
* (development) Add TP1 and TP2
* Initial commit
```

Exercise 4 â€” Git Flow: Rebase and Reset *(20 min)*

Problem statement: Simulate a simplified git-flow:

1. Create a branch `feature/avg-calculation` from `development`
2. Add `average.py` (simple function), commit
3. Before merging, `master` has advanced (simulate a commit)
4. Rebase `feature/avg-calculation` onto `master`

Solution:

```
# 1. Create the feature branch
git checkout development
git checkout -b feature/avg-calculation

# 2. Create and commit the function
echo "def average(grades): return sum(grades)/len(grades)" > average.py
git add average.py && git commit -m "Add average function"

# 3. Simulate a commit on master
git checkout master
echo "Additional file" > note.txt
git add note.txt && git commit -m "Add note.txt"

# 4. Rebase
git checkout feature/avg-calculation
git rebase master
# Feature commits are replayed on top of master

# Alternative: git merge master (classic merge)
```

Tree before rebase:

```
* (master) Add note.txt
| * (HEAD -> feature/avg-calculation) Add average function
|/
* (development) Add TP1 and TP2
| ...
```

Tree after rebase:

```
* (HEAD -> feature/avg-calculation) Add average function
* (master) Add note.txt
| ...
```

Exercise 5 â€” Full Collaborative Workflow *(15 min)*

Problem statement: Write the complete sequence of commands for the following scenario:

Two developers (Ali and Sara) work on the same GitHub repository `ens-project`.

1. Ali creates the repository, adds a README, commits, pushes to `main`

2. Sara clones the repository
3. Ali creates a branch `feature/auth` and works on it
4. Sara creates a branch `feature/ui` and works on it
5. Ali finishes first, merges into `main` via Pull Request
6. Sara rebases her branch onto `main`, then Pull Request

Solution:

```
# 1. Ali: remote repository
# (Created on github.com/ali/ens-project)
git init
echo "# ENS Project" > README.md
git add README.md && git commit -m "Initial"
git remote add origin https://github.com/ali/ens-project.git
git push -u origin main

# 2. Sara: clone
git clone https://github.com/ali/ens-project.git

# 3. Ali: feature/auth
git checkout -b feature/auth
echo "def login(): pass" > auth.py
git add auth.py && git commit -m "Add auth"
git push -u origin feature/auth
# â†’ Pull Request on GitHub

# 4. Sara: feature/ui
git checkout -b feature/ui
echo "<h1>Welcome</h1>" > index.html
git add index.html && git commit -m "Home page"
git push -u origin feature/ui

# 5. Ali: Pull Request merged into main
git checkout main
git merge feature/auth
git push origin main

# 6. Sara: rebase onto main
git checkout main
git pull origin main
git checkout feature/ui
git rebase main
# Resolve any conflicts
git push -f origin feature/ui
# â†’ Pull Request
```

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Initialization and first commit	20 %	Unable to init or commit	Steps completed with help	Init, add, commit maste
Branch management	20 %	Cannot create a branch	Creates but does not switch properly	Creation, switching, nav
Merge and conflict resolution	20 %	Cannot merge	Simple merge without conflict	Conflict resolution mast
Rebase and reset	20 %	Concepts not understood	Rebase executed with help	Rebase and reset unde
Full collaborative workflow	20 %	Workflow not mastered	Partially understood	Clone, branch, PR, reba

Pedagogical Note ENS

Teaching transposition: Versioning concepts can be introduced in middle school through text modification tracking exercises (e.g., collaborative writing). In high school, Git can be simulated with concrete team-work scenarios before using the terminal.

Trainer advice: Use analogies (e.g., "Git is like a video game save system") and prioritize practice on local repositories before introducing remotes. Merge conflicts are the trickiest step; prepare with a paper-based exercise.

Extension: Propose a group project where each student contributes on a different branch, with code review via Pull Requests on GitHub.

Conceptual Diagrams

Diagram 1 â€” Git Workflow

```
graph LR
A[Working Directory] -->|git add| B[Staging Area]
B -->|git commit| C[Local Repository]
C -->|git push| D[Remote Repository]
D -->|git clone| E[Local Clone]
C -->|git branch| F[Branch]
F -->|git merge| C
F -->|git rebase| C
style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#9b59b6,color:#fff
style F fill:#4a90d9,color:#fff
```

Diagram 2 â€” Branch Strategy

```
gitGraph
commit
branch development
checkout development
commit id: "Add TP1 and TP2"
checkout main
commit id: "Modify README"
checkout development
commit id: "Modify README dev"
checkout main
merge development id: "Resolve conflict"
branch feature/auth
checkout feature/auth
commit id: "Add auth.py"
checkout main
commit id: "Add note.txt"
checkout feature/auth
rebase main id: "Rebase onto main"
```

Session Schedule â€” 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome	5 min	5 min
Exercise 1 â€” Init / first commit	15 min	20 min
Exercise 2 â€” Branches	20 min	40 min

Exercise 3 â€” Merge + conflict	20 min	60 min
Exercise 4 â€” Rebase and reset	20 min	80 min
Exercise 5 â€” Full workflow	10 min	90 min
Total	90 min	

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_08.pdf`

Computer Science Tutorial â€” ENS Year 2 â€” Session 08 (EN)

Lecturer: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 2 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Exercise 1 â€” Analyzing a Scratch Exercise *(20 min)*

Problem statement: A teacher proposes the following Scratch exercise to 1st year middle school students:

"The Scratch cat must move forward 50 steps, say 'Hello!' for 2 seconds, then move backward 50 steps."

1. Which Scratch blocks are needed?

2. What are the cognitive difficulties for a beginner student?

3. Propose an adaptation for a struggling student.

Solution:

1. **Required blocks:**

• `move 50 steps` (motion block)

• `say "Hello!" for 2 seconds` (looks block)

• `move -50 steps` or `move 50 steps` (backward)

• `when green flag clicked` (trigger)

2. **Cognitive difficulties:**

• **Negative numbers:** students may not understand that -50 steps = moving backward

• **Sequencing:** respecting instruction order (sequential reading)

• **Temporal concepts:** "for 2 seconds" is a wait time, not execution duration

• **Abstraction:** the screen represents a mathematical space (coordinates)

3. **Adaptation (difficulty):**

• Simplified version without negative numbers:

`

move 50 steps

say "Hello!" for 2 seconds

wait 2 seconds

move 50 steps (to the right)

`

• Use colors for each block type

• Label each block with a French instruction

Exercise 2 â€” Designing a Lesson Plan *(25 min)*

Problem statement: Design a **lesson plan** for an introductory Scratch programming session for 1st year middle school students (11-12 years old). Duration: 1h00.

The plan must include:

• General objective

• Specific objectives (3)

• Prerequisites

• Materials

• Procedure (4 phases, with durations)

• Assessment

• Extension

Solution:

```
LESSON PLAN â€” Introduction to Scratch

Level: 1st Year Middle School
Duration: 60 minutes
Teacher: [Name]

---

GENERAL OBJECTIVE:
Create a first simple Scratch program
that moves a sprite.

SPECIFIC OBJECTIVES:
1. Identify the Scratch interface (stage, sprite, blocks)
2. Assemble blocks to create a sequence of instructions
3. Run the program and observe the result

PREREQUISITES:
- Ability to use a mouse
- Basic notions: forward/backward, left/right

MATERIALS:
- 1 computer for 2 students
- Access to Scratch (online or installed)
- Printed instruction sheet
- Demonstration video (1 min)

PROCEDURE:

Phase 1 â€” Guided Discovery (15 min)
- Video: "What is Scratch?" (5 min)
- In pairs: explore the interface freely (5 min)
- Group discussion: name the main areas (5 min)

Phase 2 â€” Challenge "The Walking Cat" (20 min)
- Task: "Make the cat move 100 steps then say Hello!"
- Differentiated support:
* Level 1: sheet with drawn blocks to use
* Level 2: oral instruction only
* Level 3: add a color change

Phase 3 â€” Sharing and Analysis (15 min)
```

```
- 3 pairs present their solution at the board
- Collective analysis: "Which blocks did you use?"
- Summary: Motion, Looks, Events blocks

Phase 4 â€” Bonus Challenge (10 min)
- "Make the cat walk in a square"
- Fastest students help others (peer tutoring)

ASSESSMENT:
- Formative: teacher observation (criteria grid)
- Students show their program to the teacher
- Self-assessment: "I managed to..." (3 checkboxes)

EXTENSION:
- Take-home challenge:
"Create a simple story with two characters who talk to each other"
```

Exercise 3 â€” Creating a 3-Session Progression *(15 min)*

Problem statement: From the previous lesson plan, propose a 3-session Scratch progression for 1st year middle school students.

Session	Title	Objective	Production
1	...	...	...
2	...	...	...
3	...	...	...

Solution:

Session	Title	Objective	Production
1	Discovery	Discover the interface and create a first movement	The walking cat
2	Interactions	Use control blocks (if, repeat) and sensors	Interactive quiz
3	Project	Design a guided mini-game	"Catch the apple" game

Session 2 Details â€” "Interactive Quiz":

```
- Blocks: ask, if/else, sensors
- Example: ask a question â†’ check answer â†’ say "Well done" or "Try again"
- Advanced: score with variable
```

Session 3 Details â€” "Catch the Apple":

```
- Apple sprite falling randomly
- Control a basket with arrow keys
- Score = number of apples caught in 30 seconds
- Score variable + timer
```

Exercise 4 â€” Analyzing a Student's Work *(15 min)*

Problem statement: A student produced the following Scratch program:

```
when green flag clicked
move 10 steps
repeat 10 times
move 5 steps
```

```
say "Hello" for 1 seconds
wait 2 seconds
go to x: 0 y: 0
```

1. What does this program do?

2. Identify 3 syntax or logic errors

3. Propose the corrected version

Solution:

1. **Behavior:** The cat moves 10 steps, then repeats 10 times (move 5 steps + say Hello), then waits 2 seconds and returns to center. **However** the `repeat` has no explicit `end repeat`, so the program is poorly structured.

2. **Errors:**

• **Structure:** The `repeat` block does not enclose the following instructions (no closing). Scratch uses visual "hats", so the actual order depends on stacking. The real sequence will be: move 10 â†' repeat 10 times (move 5 + say) â†' wait â†' go to (0,0).

• **Logic:** `repeat 10 times` contains `say "Hello" for 1 seconds` â†' takes 10 seconds. The student may have wanted to say it only once.

• **Pedagogical:** total duration (10+ seconds) is too long for a visible effect.

3. **Corrected version:**

```
when green flag clicked
repeat 10 times
move 10 steps
wait 0.5 seconds
say "Arrived!" for 2 seconds
```

Exercise 5 â€" Designing a Grading Rubric *(15 min)*

Problem statement: Design a criteria-based grading rubric for the "Catch the Apple" project (Session 3). The rubric must include 4 criteria with 3 levels (Acquired, Partially acquired, Not acquired).

Solution:

Grading Rubric â€" "Catch the Apple" Project

Criterion	Not acquired (0 pts)	Partially acquired (1 pt)	Acquired (2 pts)
Interface	Project contains only one sprite	Multiple sprites but no decoration	Chosen sprites, decoration
Movement	Basket does not move	Basket moves but does not follow arrows	Basket follows left/right arr
Mechanics	Apple does not fall	Apple falls but not randomly	Apple falls from random po
Score	No score variable	Score variable but does not increment correctly	Score increments on conta

Scale:

• 7-8 pts: Excellent mastery

• 5-6 pts: Acquired

• 3-4 pts: Partially acquired

• 0-2 pts: Needs revision

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Scratch exercise analysis	20 %	No analysis	Partial block analysis	Complete analy
Lesson plan design	20 %	Incomplete or incoherent plan	Partially structured plan	Complete and p
3-session progression	20 %	No progression	Progression without links between sessions	Logical and stru
Student work analysis	20 %	Errors not identified	Partial error identification	3 errors identifie
Grading rubric design	20 %	Rubric absent or unusable	Incomplete criteria rubric	Criteria-based r

Pedagogical Note ENS

Teaching transposition: Scratch is an excellent tool for introducing programming in middle school (1st year) because it avoids syntax and allows visual manipulation. Concepts of sequence, loop, and condition are addressed intuitively.

Trainer advice: Encourage the "challenge" approach to motivate students and prioritize peer tutoring. Analyzing errors in student work is an excellent training exercise to develop the pedagogical eye of future teachers.

Extension: Ask trainees to design a complete 6-session Scratch sequence with diagnostic, formative, and summative assessment.

Conceptual Diagrams

Diagram 1 â€" Key Scratch Concepts

```
mindmap
root((Scratch))
Motion
Move
Turn
Go to
Looks
Say
Think
Switch costume
Control
Repeat
If / Else
Wait
Sensing
Ask
Touching
Variables
Score
Timer
Events
Green flag
Key pressed
```

Diagram 2 â€" Scratch Session Flow

```
graph TD
A[1h Scratch Session] --> B[Phase 1: Guided Discovery]
A --> C[Phase 2: Main Challenge]
A --> D[Phase 3: Sharing and Analysis]
A --> E[Phase 4: Bonus Challenge]
```

```
B --> F[Video + free exploration]
C --> G[The walking cat]
C --> H[Level 1: visual aid]
C --> I[Level 2: oral instruction]
C --> J[Level 3: extra challenge]
D --> K[Solution presentations]
D --> L[Collective summary]
E --> M[Peer tutoring]
style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#9b59b6,color:#fff
```

Session Schedule â€” 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome	5 min	5 min
Exercise 1 â€” Scratch Analysis	20 min	25 min
Exercise 2 â€” Lesson Plan	25 min	50 min
Exercise 3 â€” 3-Session Progression	15 min	65 min
Exercise 4 â€” Student Work Analysis	15 min	80 min
Exercise 5 â€” Grading Rubric	10 min	90 min
Total	90 min	

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_09.pdf

Computer Science Tutorial â€” ENS Year 2 â€” Session 09 (EN)

Lecturer: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 2 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Exercise 1 â€” Defining the Specifications *(20 min)*

Problem statement: You need to develop a **grade management application** for your department. Write the following sections of the specifications document:

1. **Context and objectives** (2-3 sentences)

2. **Target users** (who uses the app?)

3. **Essential features** (5 minimum)

4. **Technical constraints** (language, database, platform)

Solution:

```
SPECIFICATIONS â€” ENS Grade Management

1. Context and objectives
The English department of ENS Mostaganem wants to
computerize student grade tracking for computer science
modules. The goal is to replace Excel spreadsheets
with a central application.

2. Target users
- Admin: user management, subjects, semesters
- Teacher: grade entry, average consultation
- Student (read-only): consultation of their transcript

3. Essential features
- Authentication (login/password)
- CRUD for students, teachers, subjects
- Grade entry with coefficient
- Automatic average calculation (by subject, semester)
- PDF transcript generation
- CSV export

4. Technical constraints
- Language: Python (Django) or Java (Spring Boot)
- Database: PostgreSQL or MySQL
- Interface: Responsive web (Bootstrap)
- Deployment: ENS server or cloud (Institutional hosting)
```

Exercise 2 â€” UML Design: Class Diagram *(25 min)*

Problem statement: Based on the previous specifications, design the UML class diagram for the grade management application.

Classes to identify:

• `User` (abstract class)

- `Admin`, `Teacher`, `Student` (inheritance from User)
- `Subject`, `Assessment`, `Grade`
- Relationships and multiplicities

Solution:

```
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”□
â”‚ User â”‚ (abstract)
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”‚ - id: int â”‚
â”‚ - last_name: string â”‚
â”‚ - first_name: stringâ”‚
â”‚ - email: string â”‚
â”‚ - password: string â”‚
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”‚ + login() â”‚
â”‚ + get_info() â”‚
â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¬â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
â”‚ <inheritance>
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¼â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
€â”€â”€â”€â”€â”€â”□
â”‚ â”‚ â”‚
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”´â”€â”€â”€â”€â”€â”€â”□
â”Œâ”€â”€â”€â”€â”€â”€â”´â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”□
â”Œâ”€â”€â”€â”€â”´â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”□
â”‚ Admin â”‚ â”‚ Teacher â”‚ â”‚ Student â”‚
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”‚ + manage_users() â”‚ â”‚ - subjects:list â”‚ â”‚ - group: string â”‚
â”‚ + configure() â”‚ â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜ â”‚ + enter_grade() â”‚ â”‚
+ view_grades() â”‚
â”‚ + view_average()â”‚ â”‚ + get_transcriptâ”‚
â””â”€â”€â”€â”€â”€â”€â”¬â”€â”€â”€â”€â”€â”€â”€â”€â”˜
â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
â”‚
â”Œâ”€â”€â”€â”€â”€â”€â”€â”´â”€â”€â”€â”€â”€â”€â”€â”€â”□
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”□
â”‚ Subject â”‚ â”‚ Assessment â”‚
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”‚ - id: int â”‚ â”‚ - id: int â”‚
â”‚ - title: str â”‚ â”‚ - date: date â”‚
â”‚ - credit: int â”‚ â”‚ - type: string â”‚
â”‚ - coef: float â”‚ â”‚ (Quiz,Exam,TP) â”‚
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”‚ + get_average() â”‚1 *â”‚ + get_average() â”‚
â””â”€â”€â”€â”€â”€â”€â”€â”¬â”€â”€â”€â”€â”€â”€â”€â”˜â–„â”€â”€â”€â”€â”€â”€â”˜ â”‚
â”‚ â”Œâ”€â”€â”€â”€â”€â”€â”´â”€â”€â”€â”€â”€â”€â”□
â”‚ â”‚ Grade â”‚
â”‚ â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
â”‚ â”‚ - value: floatâ”‚
â”‚ â”‚ - coeff: floatâ”‚
â”‚ â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
```

Main relationships:

- `Subject` 1 â”€â”€ * `Assessment` (one subject has several assessments)
- `Assessment` 1 â”€â”€ * `Grade` (one assessment has several grades)

• `Student` 1 â”€â”€ * `Grade` (one student has several grades)

• `Teacher` *â”€â”€* `Subject` (one teacher can have several subjects)

Exercise 3 â€” Architecture and Routes *(15 min)*

Problem statement: For the web application, define:

1. The technical architecture (3-tier)

2. REST API routes (method + URL)

3. Minimal views/templates

Solution:

1. 3-tier Architecture:

```
[Browser Client]
â†• HTTP / HTTPS
[Backend Django / Spring]
â†• JDBC / ORM
[PostgreSQL Database]
```

2. REST API Routes:

Method	URL	Function
POST	/api/login	Authentication
GET	/api/students	Student list
GET	/api/students/{id}	Student details
POST	/api/students	Create student
GET	/api/subjects	Subject list
GET	/api/subjects/{id}/average	Subject average
POST	/api/grades	Enter grade
GET	/api/students/{id}/transcript	PDF transcript
GET	/api/export/csv	CSV export

3. Minimal views:

• Login page (`login.html`)

• Teacher dashboard (`dashboard.html`)

• Grade entry (`grade_entry.html`)

• Student transcript (`transcript.html`)

• Admin panel (`admin.html`)

Exercise 4 â€” Wireframe Mockup *(15 min)*

Problem statement: Draw (in text) the wireframe of the **grade entry page** for a teacher. The page must contain:

• Navigation bar (top)

• Subject + group selector

• Table: Last names | First names | Grade

• "Save" button

• Footer with active session

Solution:

```
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”
€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â
”€â”€â”□
â”‚ [Logo] ENS Grade Management Welcome, Dr Belacel [Logout] â”‚
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”
€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â
”€â”€â”¤
â”‚ â”‚
â”‚ Grade Entry â”‚
â”‚ â”‚
â”‚ Subject: [OOP ________________] Group: [ENS2-A ___] â”‚
â”‚ â”‚
â”‚ â”Œâ”€â”€â”€â”€â”€â”€â”€â”¬â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¬â”€â”€â”€â”€â”€â”€â”€â
”€â”¬â”€â”€â”€â”€â”€â”€â”€â”□ â”‚
â”‚ â”‚ No â”‚ Last Nameâ”‚ First Nameâ”‚ Grade â”‚ â”‚
â”‚ â”œâ”€â”€â”€â”€â”€â”€â”€â”¼â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¼â”€â”€â”€â”€â”€â”€â”€â
”€â”¼â”€â”€â”€â”€â”€â”€â”€â”¤ â”‚
â”‚ â”‚ 1 â”‚ Benali â”‚ Sara â”‚ [15.00]â”‚ â”‚
â”‚ â”‚ 2 â”‚ Ziani â”‚ Ahmed â”‚ [12.50]â”‚ â”‚
â”‚ â”‚ 3 â”‚ Kaci â”‚ Leila â”‚ [17.00]â”‚ â”‚
â”‚ â”‚ 4 â”‚ ... â”‚ ... â”‚ [...] â”‚ â”‚
â”‚ â””â”€â”€â”€â”€â”€â”€â”€â”´â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”´â”€â”€â”€â”€â”€â”€â”€â
”€â”´â”€â”€â”€â”€â”€â”€â”€â”˜ â”‚
â”‚ â”‚
â”‚ [ Save ] [ Cancel ] â”‚
â”‚ â”‚
â”‚ Last saved: 15/03/2026 10:32 â”‚
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”
€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â
”€â”€â”¤
â”‚ Session: Teacher â€” ENS Mostaganem â€” 2025/2026 â”‚
â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”
€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â
”€â”€â”˜
```

Exercise 5 â€” Development Plan (Milestones) *(15 min)*

Problem statement: Establish a 6-week development plan for the application. Define 3 sprints of 2 weeks with deliverables for each sprint.

Solution:

```
DEVELOPMENT PLAN â€” Sprint 0 to 3

SPRINT 0 (Week 1-2) â€” Foundations
D1-3: Environment setup (Django/Spring + PostgreSQL)
D4-7: Data model (entity classes)
D8-10: Authentication + static pages
âœ… Deliverable: App with login + navbar

SPRINT 1 (Week 3-4) â€” Core Features
D11-13: CRUD students, teachers, subjects
D14-16: Grade entry with coefficients
D17-18: Automatic average calculation
```

```
âœ… Deliverable: Grade entry and consultation

SPRINT 2 (Week 5-6) â€" Finalization
D19-20: PDF transcript
D21-22: CSV export
D23-24: User testing + fixes
D25-26: Deployment + documentation
âœ… Deliverable: Delivered app + training

DISTRIBUTION:
- Model and back-end: 60 %
- Front-end and UI: 25 %
- Testing and deployment: 15 %
```

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Specifications document	20 %	Missing or vague sections	Partially written	Complete, clear, and s
UML class diagram	20 %	Absent or incorrect	Classes identified without relationships	Classes + inheritance
Architecture and API routes	20 %	Architecture not defined	Partial routes	3-tier architecture + co
Wireframe mockup	20 %	Absent or illegible	Draft mockup	Detailed and coherent
Development plan (sprints)	20 %	No plan	Partial milestones	3 sprints with concrete

Pedagogical Note ENS

Teaching transposition: Writing specifications and UML modeling can be transposed to high school as a central project. Students learn to structure algorithmic thinking and anticipate user needs before coding.

Trainer advice: Emphasize the importance of the design phase before development. Have students work in pairs on concrete cases (e.g., school application) to make the exercise authentic.

Extension: Propose a complete mini-project where students create the class diagram, implement the backend, and present their work in a defense.

Conceptual Diagrams

Diagram 1 â€" 3-tier Architecture

```
graph TB
subgraph Client
A[Web Browser]
end
subgraph Server
B[Django / Spring Backend]
C[HTML Templates]
D[REST API]
end
subgraph Database
E[PostgreSQL / MySQL]
end
A -->|HTTP/HTTPS| B
B --> D
B --> C
B -->|JDBC/ORM| E
style A fill:#4a90d9,color:#fff
```

```
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#9b59b6,color:#fff
```

Diagram 2 â€” Project Lifecycle

```
graph LR
A[Specifications] --> B[UML Diagram]
B --> C[API Architecture]
C --> D[Mockup]
D --> E[Sprint 1: Foundations]
E --> F[Sprint 2: Core Features]
F --> G[Sprint 3: Finalization]
G --> H[Deployment]
style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#e74c3c,color:#fff
style F fill:#e74c3c,color:#fff
style G fill:#9b59b6,color:#fff
style H fill:#2ecc71,color:#fff
```

Session Schedule â€” 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome	5 min	5 min
Exercise 1 â€” Specifications	20 min	25 min
Exercise 2 â€” Class Diagram	25 min	50 min
Exercise 3 â€” Architecture and Routes	15 min	65 min
Exercise 4 â€” Wireframe Mockup	15 min	80 min
Exercise 5 â€” Development Plan	10 min	90 min
Total	90 min	

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_2Ã¨me_annÃ©e_AN_10.pdf`

Computer Science Tutorial â€” ENS Year 2 â€” Session 10 (EN)

Lecturer: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 2 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Session Objective

Finalize the grade management project started in the previous session:

- **Presentation by each group** (5 min)
- **Functional demonstration** (3 min)
- **Questions and peer evaluation** (2 min)
- **Source code and documentation submission**

Exercise 1 â€” Code Finalization (20 min)

Each pair has 20 minutes to finalize their application before the presentation.

Finalization Checklist:

Task	Done
Functional CRUD students (create, read)	â˜
Grade entry with coefficients	â˜
Average calculation (simple + weighted)	â˜
At least 3 students + 2 subjects in database	â˜
Responsive interface (CSS)	â˜
Error handling (grade > 20, empty fields)	â˜

Self-assessment questions:

- Does my application handle edge cases (negative grade, student without grades)?
- Do data persist between sessions?
- Is the interface usable on mobile?

Exercise 2 â€” Oral Presentation (25 min)

Presentation Schedule

Group	Slot
Group 1 â€” Benali/Ziani	00:00 â†’ 05:00
Group 2 â€” Kaci/Mokhtar	05:00 â†’ 10:00

Group 3 â€" Amine/Fatima	10:00 â† 15:00
Group 4 â€" Youcef/Leila	15:00 â† 20:00
Group 5 â€" Hocine/Nadia	20:00 â† 25:00

Presentation Structure (5 min)

```
1. Project introduction (30 sec)
- "Our application allows..."

2. Architecture and technical choices (1 min)
- Language, framework, database
- "We chose Django because..."

3. Demonstration (2 min)
- Add a student
- Enter grades
- Display the average

4. Difficulties encountered (1 min)
- "The hardest part was..."
- "We solved it by..."

5. Conclusion and perspectives (30 sec)
- "If we had more time, we would add..."
```

Peer Evaluation Rubric

Criterion	0	1	2
Functional application	No demo	Partial	Complete
Structured code	Messy	Readable	Clean + commented
Oral presentation	Inaudible	Clear	Captivating
Error handling	None	Partial	Complete

Exercise 3 â€" Peer Evaluation (15 min)

Problem statement: For each group (except yours), fill in the evaluation rubric below.

Evaluation Form:

```
GROUP EVALUATED: _______________

Criteria (1-4):
- Functionality: ___ /4
- User interface: ___ /4
- Code quality: ___ /4
- Presentation: ___ /4

Strengths:
- _____________________________
- _____________________________

Areas for improvement:
- _____________________________
- _____________________________

Final grade: ___ /20
```

Exercise 4 â€” Project Retrospective (15 min)

Problem statement: In pairs, write a retrospective (format "Keep, Problem, Try") of 10 lines maximum.

Keep â€” What went well:

- _____
- _____
- _____

Problem â€” Difficulties encountered:

- _____
- _____
- _____

Try â€” What you would do differently:

- _____
- _____
- _____

Example retrospective:

```
KEEP:
- Pair work allowed task division (front-end / back-end)
- Django makes CRUD easy with models

PROBLEM:
- Grade management with coefficients was complex (average calculation)
- We underestimated deployment time

TRY:
- Create a class diagram before coding
- Allow more time for user testing
```

Exercise 5 â€” Submission and Documentation (15 min)

Problem statement: Create a `README.md` file for the project. It must contain:

1. Project title
2. Description (2-3 lines)
3. Prerequisites (Python, dependencies)
4. Installation and execution (3-4 commands)
5. Project structure (file tree)
6. Authors

Solution:

```
# GradeManageENS

Web application for grade management for the English
department of ENS Mostaganem.
```

```
## Prerequisites

- Python 3.10+
- Django 4.2+
- PostgreSQL 14+

## Installation
```

git clone https://github.com/username/grade-manage-ens.git

cd grade-manage-ens

python -m venv venv

source venv/bin/activate # Windows: venv\Scripts\activate

pip install -r requirements.txt

python manage.py migrate

python manage.py runserver

```
## Structure
```

grade-manage-ens/

â”œâ”€â”€ manage.py

â”œâ”€â”€ requirements.txt

â”œâ”€â”€ README.md

â”œâ”€â”€ app/

â”, â”œâ”€â”€ models.py # Classes: User, Student, Subject, Grade

â”, â”œâ”€â”€ views.py # Routes and controllers

â”, â”œâ”€â”€ urls.py # URL configuration

â”, â”œâ”€â”€ templates/ # HTML (base, login, dashboard, entry, transcript)

â”, â”""â”€â”€ static/ # CSS, JS

â”œâ”€â”€ notes/

â”, â”œâ”€â”€ admin.py

â”, â”""â”€â”€ tests.py

â""â”€â”€ db.sqlite3

```
## Authors

- BENALI Sara â€" ENS Mostaganem
- ZIANI Ahmed â€" ENS Mostaganem
```

Project Grading Scale

Criterion	Weight	Points
Functionality (CRUD, entry, average)	40 %	/8
Code quality (structure, naming, readability)	20 %	/4

User interface (HTML/CSS, responsive)	15 %	/3
Oral presentation + demo	15 %	/3
Documentation (README, retrospective)	10 %	/2
Total	100 %	/20

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Code finalization (CRUD, grades, averages)	20 %	Application not functional	Partial features	CRUD, entry,
Oral presentation and demo	20 %	Absent or incoherent presentation	Partial presentation	Clear structur
Peer evaluation	20 %	No evaluation submitted	Incomplete evaluations	Grids filled wi
Project retrospective (KPT)	20 %	Absent or superficial	Partially written	Relevant ana
Documentation and README	20 %	Missing or incomplete	Partial README	Complete RE

Pedagogical Note ENS

Teaching transposition: Project defense is transposable to high school as an oral presentation where students present their website or Scratch game. This develops transversal skills (communication, critical thinking, self-assessment).

Trainer advice: Structure presentations with limited time to get students used to professional constraints. The peer evaluation rubric builds responsibility and develops critical thinking.

Extension: Organize a "mini-competition" where an external jury (other teachers, professionals) evaluates projects and awards prizes.

Conceptual Diagrams

Diagram 1 â€" Presentation Flow

```
graph TD
A[5 min Defense] --> B[1. Project Introduction]
A --> C[2. Architecture and Technical Choices]
A --> D[3. Demonstration]
A --> E[4. Difficulties Encountered]
A --> F[5. Conclusion and Perspectives]
B --> G["30 sec"]
C --> H["1 min"]
D --> I["2 min"]
E --> J["1 min"]
F --> K["30 sec"]
style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#9b59b6,color:#fff
style F fill:#4a90d9,color:#fff
```

Diagram 2 â€" Project Evaluation

```
mindmap
root((Project Evaluation))
Functionality
CRUD
```

```
Grade entry
Average calculation
Error handling
Code Quality
Structure
Naming
Readability
User Interface
HTML/CSS
Responsive
Accessibility
Presentation
Clarity
Demonstration
Time management
Documentation
README
Retrospective
Commented code
```

Session Schedule â€” 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome + instructions	5 min	5 min
Exercise 1 â€” Finalization	20 min	25 min
Exercise 2 â€” Presentations (5 groups Ã— 5 min)	25 min	50 min
Exercise 3 â€” Peer Evaluation	15 min	65 min
Exercise 4 â€” Retrospective	15 min	80 min
Exercise 5 â€” README + submission	10 min	90 min
Total	90 min	